Red-black trees

Part III Data Structures

Scope

Black height and local color constraints guarantee logarithmic height. Exact NIL and parent-child wiring supports proofs of the three insertion cases, four deletion cases, and their mirrors. Before-and-after figures use the same keys to check extra black, root cases, and record identity.

Prerequisites

Binary search trees, rotations, and invariant proofs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Define a nearly balanced BST using red-black properties.
- Prove that rotations preserve inorder.
- Trace insertion and deletion repair and verify black height.
- 13.1 Properties of red-black trees
- 13.2 Rotations
- 13.3 Insertion
- 13.4 Deletion

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Read the roles and path counts first

Dark nodes are black B; pale red nodes are red R. B/R after each key also identifies its color. An orange outline marks the current repair position; it is not a third node color. Rotation diagrams leave colors unspecified.

Operation	Roles in the figures
Rotation	x: old local root; y: right child; p: old parent of x; α, β, γ: subtrees preserving in-order intervals
Insertion	z: current red node; p: parent; g: grandparent; u: uncle
Deletion	x: deficit position; p: parent; w: sibling; near: nephew toward x; far: nephew away from x

x +1 adds one black only for the proof. Figure path counts **include the starting node and terminal NIL**; $bh(x)$ excludes x. Omitted empty children are black NIL. A NIL near/far nephew is shown explicitly.

Properties of red-black trees

Color augments a BST to bound height

Internal nodes store key, data, left, right, parent, and color. Examples use distinct keys; duplicate insertion returns the existing record. Keeping duplicate keys instead requires a strict total order on (key,unique ID); the BST arguments remain valid.

The five properties are: (1) every node is red or black; (2) the root is black; (3) every external NIL leaf is black; (4) both children of a red node are black; (5) all paths from any node to descendant NIL leaves contain the same number of black nodes. Property5 must hold in every subtree, not merely for two selected whole-tree paths.

One black sentinel NIL represents all missing children and the root's parent. An empty tree has root=NIL. NIL has no valid key/data; NIL.left=NIL.right=NIL makes color reads safe. Non-NIL child links must point back to their correct parent. All empty links share one object, so they cannot each have an independent NIL.parent.

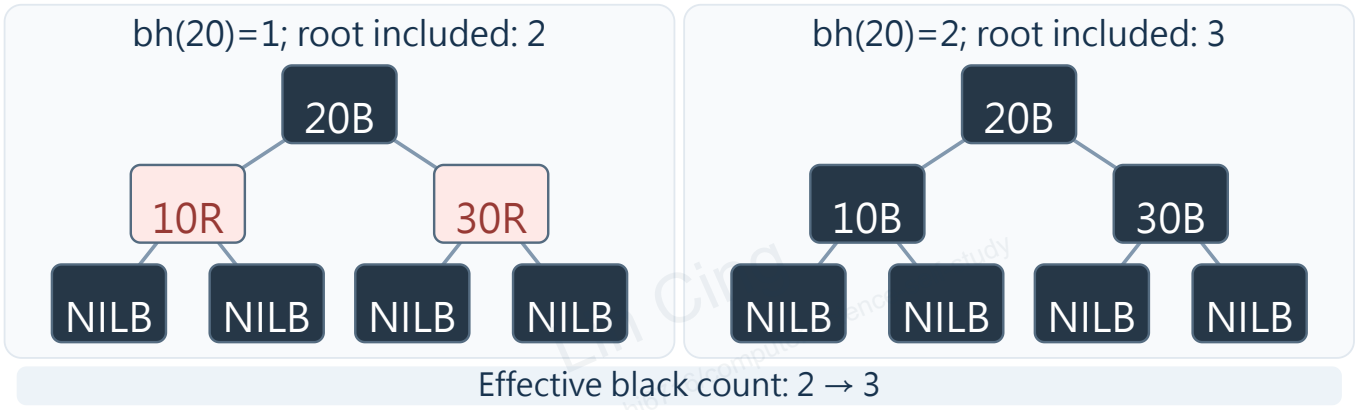
Black height excludes the start and includes terminal NIL

$bh(x)$ counts black nodes on any path from x to a descendant NIL, **excluding x and including terminal NIL**; $bh(NIL) = 0$. For a red child, $bh(child) = bh(x)$; for a black child, $bh(child) = bh(x) - 1$. The parent's count includes the child only when black.

With root20B and children10R,30R, all other links NIL, bh(20)=1 and bh(10)=bh(30)=1, while bh(NIL)=0. Making both children black gives bh(20)=2 and bh(10)=bh(30)=1. Complete black counts including the root rise from2 to3, but each bh still excludes its own starting node.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Black height excludes the start and includes terminal NIL (continued)



Here h is the longest root-to-NIL edge count, with empty h=0 and singleton black-root h=1. Counting edges only between internal nodes instead gives one less for a nonempty tree. Do not exchange conventions within a formula.

Minimum node count and the full height induction

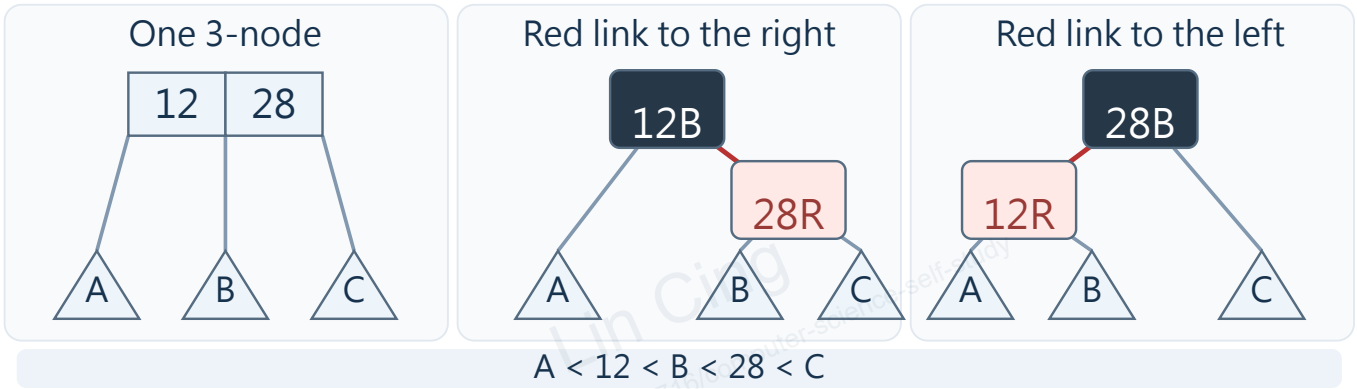
Lemma: a subtree rooted at x has at least $2^{bh(x)} - 1$ internal nodes. Induct on height. At NIL, 0=2⁰−1. Both children of an internal x have smaller height and black height at least bh(x)−1. Since bh(x)≥1, each has at least $2^{bh(x)-1} - 1$ internal nodes. Adding x gives

$$n_x \geq 2(2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1.$$

A longest root-NIL path has B black and R red nodes. Root and NIL are black and reds are nonadjacent. Pair each red with a distinct preceding black, giving R≤B−1. The path has h+1 nodes, so h+1=B+R≤2B−1 and h≤2(B−1)=2bh(root). Combining the lemma gives $n \geq 2^{h/2} - 1$, hence $h \leq 2\log_2(n+1)$. The empty tree also satisfies0≤0.

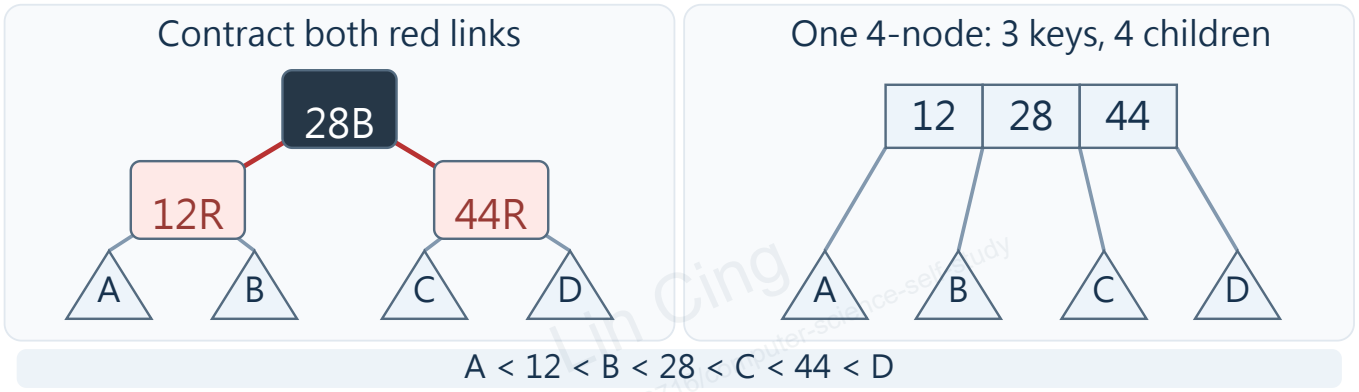
Search, min, max, successor, and predecessor follow paths in O(log(n+1)). Insertion and deletion still require repair-cost proofs. Complete inorder, preorder, and postorder output all n nodes in Θ(n). Red-black trees do not require child heights to differ by at most1; that is not their definition.

Why can one 3-node have two red-black encodings?



Merge a red child with its black parent into one multi-key node. A lone black node is a 2-node. A,B,C have black roots (possibly NIL) and equal black height; the inequalities apply to every key in each subtree. Both encodings preserve in-order sequence. **General red-black trees need not lean left.**

Equal black counts become equal external-leaf depths



No consecutive red links limits each merged node to three keys: a 2-, 3-, or 4-node. Boundary intervals stay ordered. Subtracting terminal NIL's black, each path's black count equals its multi-key-node count after contraction. Equal counts give equal external-leaf depths. Incoming links take the child's color; the root is separately black.

Rotations

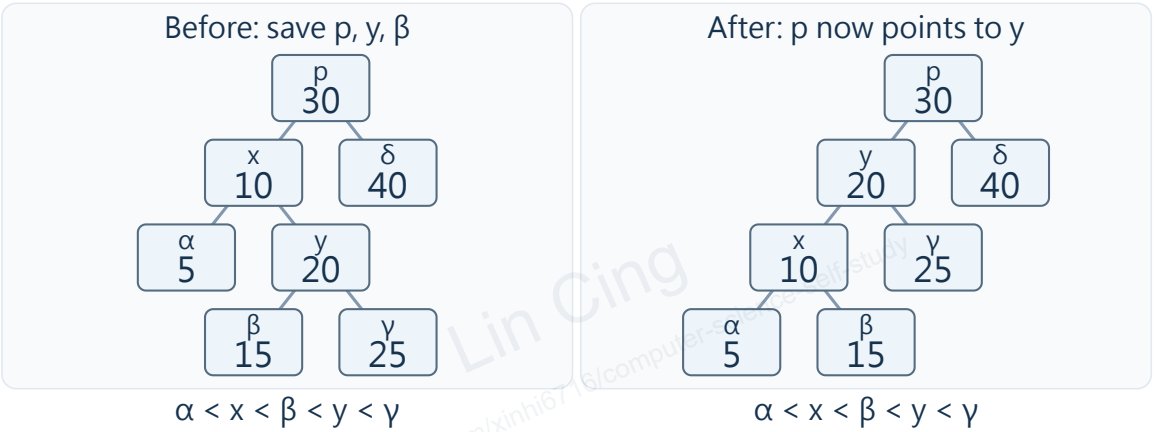
Save the old parent before reconnecting the local root

Assume x≠NIL and y=x.right≠NIL. Save p=x.parent and β=y.left. Internal links in α=x.left and γ=y.right remain unchanged.

```
LEFT-ROTATE(T,x):
    p=x.parent; y=x.right; beta=y.left
    x.right=beta
    if beta!=NIL: beta.parent=x
    y.parent=p
    if p==NIL: T.root=y
    elif x==p.left: p.left=y
    else: p.right=y
    y.left=x
    x.parent=y
```

Update **x's old parent p to point to y**. y's old parent is x; do not confuse them. Do not write beta.parent when beta=NIL, because that could overwrite the shared sentinel context used by deletion repair. Rotation changes no color, key, or data.

Rotations (continued)



Pointer table and the complete right mirror

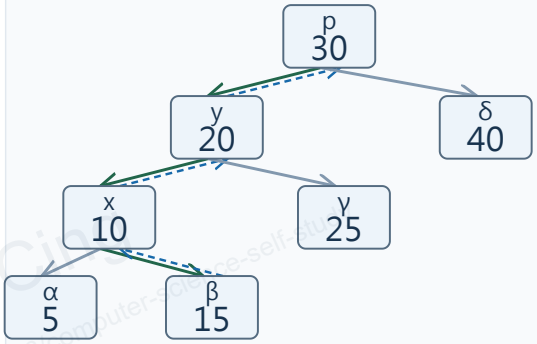
For p30,x10,y20,β15 with x=p.left, write10.right=15,15.parent=10,20.parent=30,30.left=20,20.left=10,10.parent=20. If p=NIL, replace the30.left write with root=20 and set20.parent=NIL.

```
RIGHT-ROTATE(T,y):
    p=y.parent; x=y.left; beta=x.right
    y.left=beta
    if beta!=NIL: beta.parent=y
    x.parent=p
    if p==NIL: T.root=x
    elif y==p.right: p.right=x
    else: p.left=x
    x.right=y
    y.parent=x
```

Pointer table and the complete right mirror (continued)

1. 10.right = 15
2. 15.parent = 10
3. 20.parent = 30
4. 30.left = 20
5. 20.left = 10
6. 10.parent = 20

Solid ↓ : child
Dashed ↑ : parent



If p=NIL: step 4 sets root=20; step 3 sets 20.parent=NIL.

Rotation preserves order, not arbitrary color invariants

Before and after left rotation, inorder is α,x,β,y,γ. Keys in α<x, keys in β lie between x and y, and γ>y, so β remains valid as x.right. Outside the local region, p's same-side link changes from x to y and all bounds remain. Non-NIL β.parent, x.parent, and y.parent become x,y,p respectively, keeping structure consistent. Right rotation is symmetric.

Only constantly many fields change, giving O(1) time. Finding the repair position still takes O(log n), so a constant-time rotation does not imply constant-time insertion or deletion. Rotation preserves BST order and record identities, **not automatically black height or the no-red-red rule**. Recoloring needs the local black-count arguments below.

Insertion

A red leaf can violate only root color or red adjacency

Use BST search to find an empty position. Initialize z.left=z.right=NIL and z.color=R, then attach its parent. Replacing a black NIL with a red leaf retains the same terminal black NIL, so path black counts do not change. Inserting black instead would add one black only on paths through that position, violating property5.

The first inserted root is temporarily red and becomes black at the end. Otherwise only red z and red parent p may conflict. The repair-loop invariant says z is red; if its parent is root, that parent is black; at most one violation exists, either a red root or z-parent red adjacency; all other color properties and BST order hold. Entering with p red implies p is not root, so grandparent g exists and is black, or another red-red violation would already exist.

Complete insertion repair

The first branch has p=g.left. The other exchanges left/right and LEFT/RIGHT-ROTATE everywhere.

```
while z.parent.color==R:
    if z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        if u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        else:
            if z==z.parent.right:              // I2, followed by I3
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                   // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    else: execute the fully mirrored three branches
T.root.color=B
```

Complete insertion repair (continued)

I2 and I3 are not mutually exclusive. I2 must be followed by I3. Use the updated z.parent and grandparent, not a stale pre-rotation p.

When p=g.right	Explicit action
I1: uncle g.left is red	Make p/uncle black and g red; z=g
I2: black uncle, z=p.left	z=p; RIGHT-ROTATE(z), then I3
I3: black uncle, outer child	New z.parent black, grandparent red; LEFT-ROTATE(grandparent)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I1 preserves complete black counts

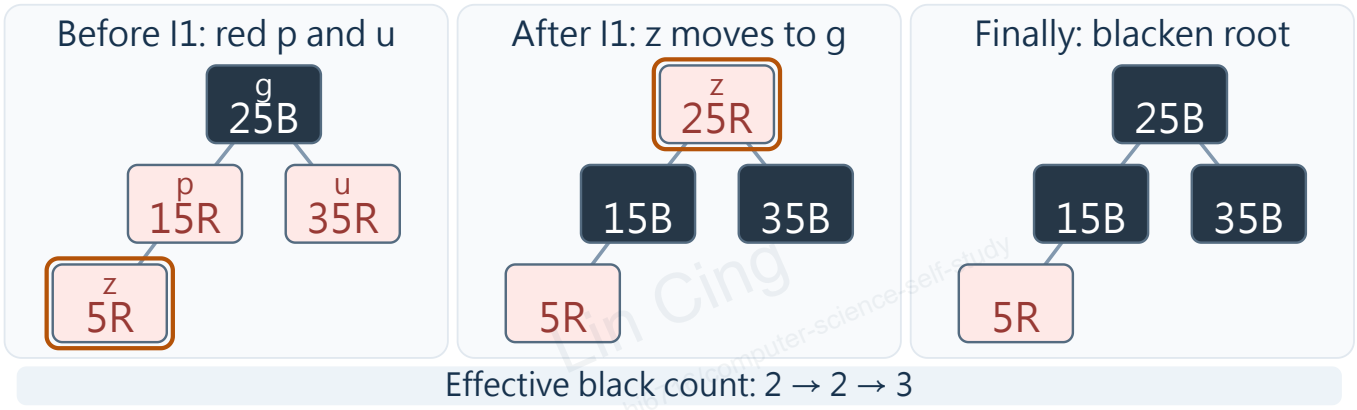
Suppose every path below p and uncle u contributes b black nodes, including terminal NIL. Initially g is black and p/u red, giving1+b on every complete path through g. Making g red and p/u black retains1+b. Although bh(g), which excludes g, changes from b to b+1, **complete ancestor paths through g gain no black node**.

Set new z=g. Red adjacency can only move up to g and its parent; the original z-p conflict is removed by making p black. If g becomes root, only the root is temporarily red. Blackening it at the end adds one black consistently to every whole-tree path.

Insert25,15,35,5. New5R has parent15R, uncle35R, grandparent25B. I1 makes15/35 black and25 red, then z=25. Its parent NIL is black, so terminate and blacken root25. A nonroot g does not receive this final whole-tree increase.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I1 preserves complete black counts (continued)

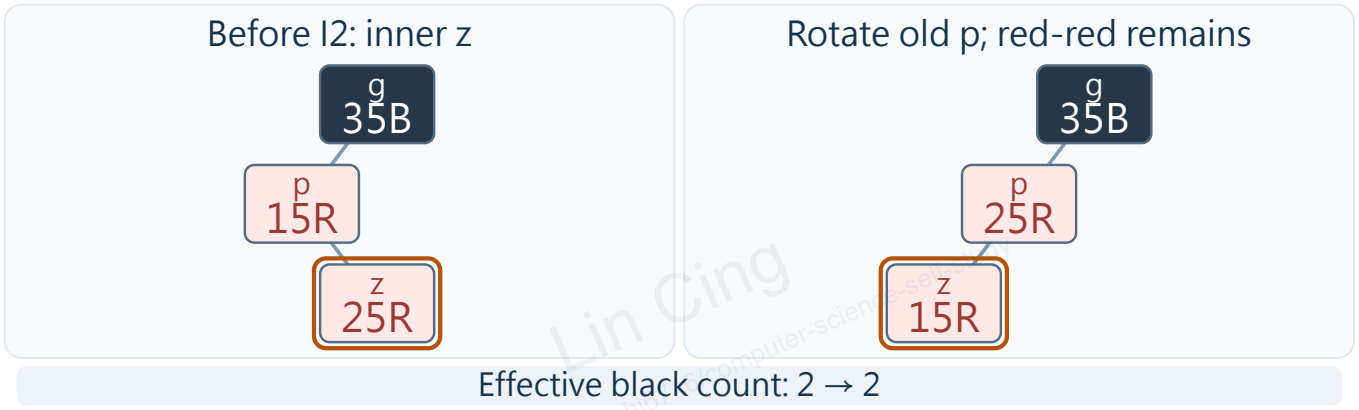


I2 converts an inner bend to an outer chain

With black uncle, p=g.left, and z=p.right, first set z=p and left-rotate the old p. Both rotated internal nodes are red, so rotation changes no black contribution to boundary paths and preserves inorder. Old p becomes a left red child under the new local root, producing an outer chain that still requires I3.

Insert35,15,25. Before repair, root35B has left15R with right25R. Left-rotating15 leaves root35B,35.left=25R,25.left=15R, and the algorithm's z now points to15. The15/25 red conflict remains; do not return here.

I2 converts an inner bend to an outer chain (continued)

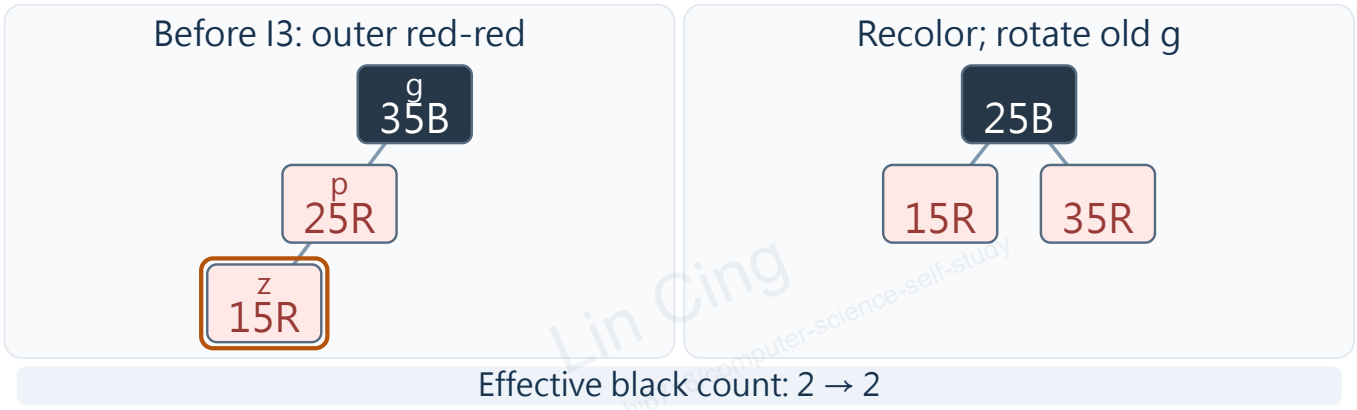


I3 recolors and rotates to restore every local path

The outer chain is gB-left pR-left zR, with black uncle. Make p black and g red, then right-rotate g to make p the local root. Let α,β be z's children, γ the old p.right, and δ the old g.right. All four boundary subtrees have black roots, possibly NIL, and contribute b black nodes per complete downward path. Each original path through g contributes1+b; after repair, each path through new p also contributes1+b. z remains red under black p, while red g has black γ,δ children. New local root p is black and cannot conflict with its external parent.

Continue the example: make25 black and35 red, right-rotate35, and obtain25B with15R,35R children. Inorder15,25,35 is preserved, bh(root)=1, and every root-NIL path contains2 blacks. Mirror insertion15,35,25 first right-rotates35, then left-rotates15, giving the same tree.

I3 recolors and rotates to restore every local path (continued)



Each I1 moves z up two levels without rotation. I2+I3 use at most two rotations and terminate. Recoloring is O(log(n+1)), rotations number at most2, and total insertion including BST search is O(log(n+1)).

Deletion

Distinguish z, y, x, and y's original color

z is the record to remove. If z has fewer than two children, y=z; otherwise y is the minimum in z.right and moves as a whole object into z's position. x is the sole possible child filling **y's old position**, namely y.right for a successor with no left child. Save yOriginalColor before giving moved y z.color. Missing black depends on yOriginalColor, not y's new color or merely z.color.

```
RB-TRANSPLANT(u,v):
    if u.parent==NIL: root=v
    elif u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent              // write even when v==NIL
```

NIL.parent now records the parent of the current empty x position so repair can find its sibling. It is not a universal parent for all empty links. The root's parent is NIL. Rotations skip beta.parent for beta=NIL to preserve this context. After a stable operation, NIL.parent may be reset to NIL; NIL.color must always remain black.

Complete object relocation in RB-DELETE

```
RB-DELETE(z):                  // z is a found, valid record
    y=z; yOriginalColor=y.color
    if z.left==NIL:
        x=z.right; RB-TRANSPLANT(z,z.right)
    elif z.right==NIL:
        x=z.left; RB-TRANSPLANT(z,z.left)
    else:
        y=MINIMUM(z.right); yOriginalColor=y.color; x=y.right
        if y.parent==z:
            x.parent=y          // including x==NIL
        else:
            RB-TRANSPLANT(y,y.right)
            y.right=z.right; y.right.parent=y
        RB-TRANSPLANT(z,y)
        y.left=z.left; y.left.parent=y
        y.color=z.color
    if yOriginalColor==B: RB-DELETE-FIXUP(x)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete object relocation in RB-DELETE (continued)

Move the entire y object, retaining its key, data, and handle; do not give it z's data. An API may first SEARCH(key), returning NOT_FOUND without modification if absent. This procedure assumes an existing z and never treats NIL as an ordinary deletable record.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

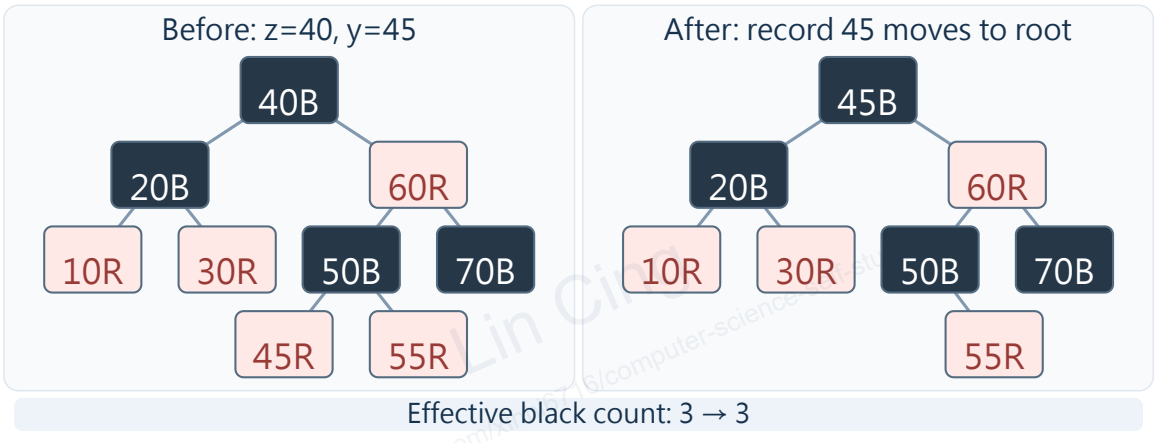
Successor relocation and NIL parent

Insert40,20,60,10,30,50,70,45,55. The tree has root40B, left20B(10R,30R), and right60R with left50B(45R,55R) and right70B. Deleting z40 chooses y45R and x=NIL, with y.parent50≠z.

Remove45 from50.left, giving50.left=NIL and NIL.parent=50. Attach60 as y.right and set60.parent=45. Replace root40 with y, then attach20 as y.left and set20.parent=45. Give y color B. Since y's original color was R, no black repair runs. Root is45B, original45's data moves with45, and only record40 is removed.

If y=z.right, x.parent must instead be set to y, never left pointing to deleted z. Both branches apply whether x is real or NIL.

Successor relocation and NIL parent (continued)



Track a missing black at x without a third color

Removing an originally red y reduces no path black count. Giving moved y z's old color preserves the other local relationships, so no fix is needed. If y was black, temporarily assign its missing black to x's position. Count one extra black on each path through x to match the pre-deletion effective counts.

This proof-only extra black does not change the actual R/B color domain. A red x absorbs it by becoming black. At the root, drop it from all tree paths together. Otherwise the loop has a black nonroot x. Its sibling w cannot be NIL: w would contribute only one ordinary NIL black while x's side contributes at least NIL plus extra, contradicting equal effective black counts.

Below, x=p.left, w=p.right, near=w.left, and far=w.right. A right-side deficit exchanges left and right, giving near=w.right and far=w.left, and reverses rotations.

Complete left-side deletion fixup

```
while x!=root and x.color==B:
    if x==x.parent.left:
        w=x.parent.right
        if w.color==R:                        // D1
            w.color=B; x.parent.color=R
            LEFT-ROTATE(T,x.parent)
            w=x.parent.right
        if w.left.color==B and w.right.color==B: // D2
            w.color=R
            x=x.parent
        else:
```

Complete left-side deletion fixup (continued)

```
            if w.right.color==B:              // D3, followed by D4
                w.left.color=B; w.color=R
                RIGHT-ROTATE(T,w)
                w=x.parent.right
            w.color=x.parent.color            // D4
            x.parent.color=B; w.right.color=B
            LEFT-ROTATE(T,x.parent)
            x=root
    else: execute the fully mirrored four branches
x.color=B
```

Refresh w after D1 and after D3. The four cases are not mutually exclusive elif branches. One iteration may execute D1→D2 or D1→D3→D4; the latter and direct D4 terminate.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete left-side deletion fixup (continued)

When x=p.right	w=p.left, near=w.right, far=w.left
D1: red w	w black, p red; RIGHT-ROTATE(p); refresh w
D2: both w children black	w red; x=p, move up or absorb by final blackening
D3: far black, near red	w.right black, w red; LEFT-ROTATE(w); refresh w
D4: far red	w takes p's old color; p and w.left black; RIGHT-ROTATE(p); x=root

Lin Cing https://github.com/xinhi6716/computer-science-self-study

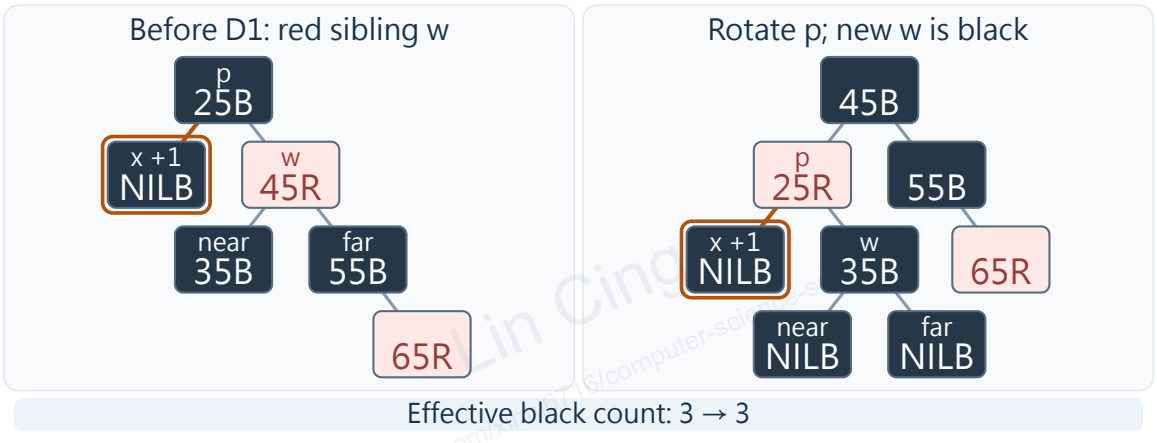
D1 converts a red sibling to a black sibling

Let x's **actual** complete black count be b, including x, or NIL if x=NIL. Extra makes it b+1. Red w forces p and both w children black. Each w-child path contributes b+1, so every effective complete path including p contributes b+2.

Make w black and p red, then left-rotate p. New black local root w has a red left child p whose x-side effective count b+1 equals the old near-child count b+1. Its right side, the old far child, also contributes b+1. Total remains b+2. Extra remains at x; its new sibling is w's old black near child. Proceed to D2, D3, or D4.

Insert15,25,35,45,55,65 and delete15. After removal x=NIL is25B.left and w45R. D1 makes45 black and25 red, then left-rotates25. New root45B has left25R(right35B) and right55B(right65R); x remains25.left. Complete black counts through ancestors are unchanged.

D1 converts a red sibling to a black sibling (continued)



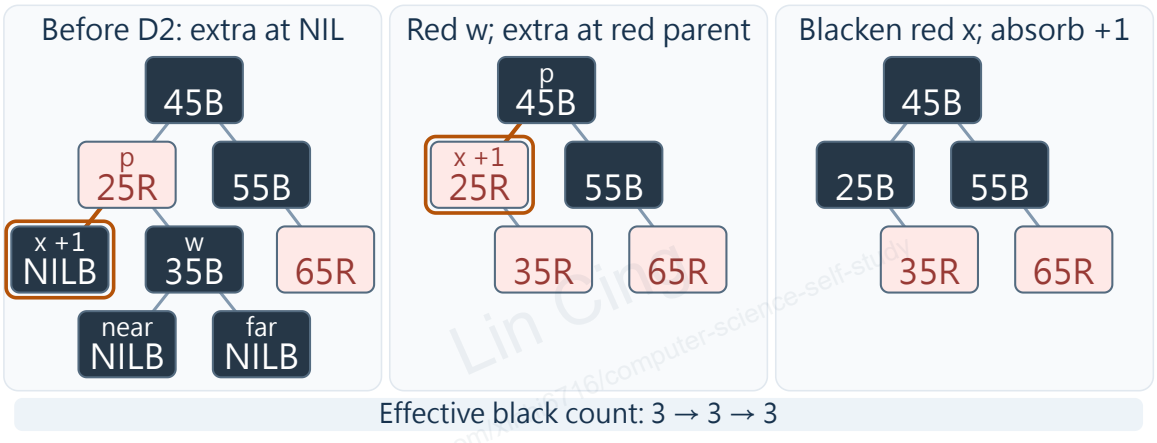
D2 moves the deficit upward or absorbs it at a red parent

Now w and both its children are black. x contributes actual b and effective b+1; w contributes b+1 and each child b. Making w red reduces its actual contribution to b. Both branches now have equal **actual** counts, but the whole p-subtree is one black short relative to its ancestors. Move extra to p by setting x=p.

If p was red, final blackening absorbs the extra and stops. If p was black and nonroot, iterate upward. At the root, drop extra consistently from all paths. D1 just made p red, so D1 followed by D2 absorbs in this round and cannot climb to another D1.

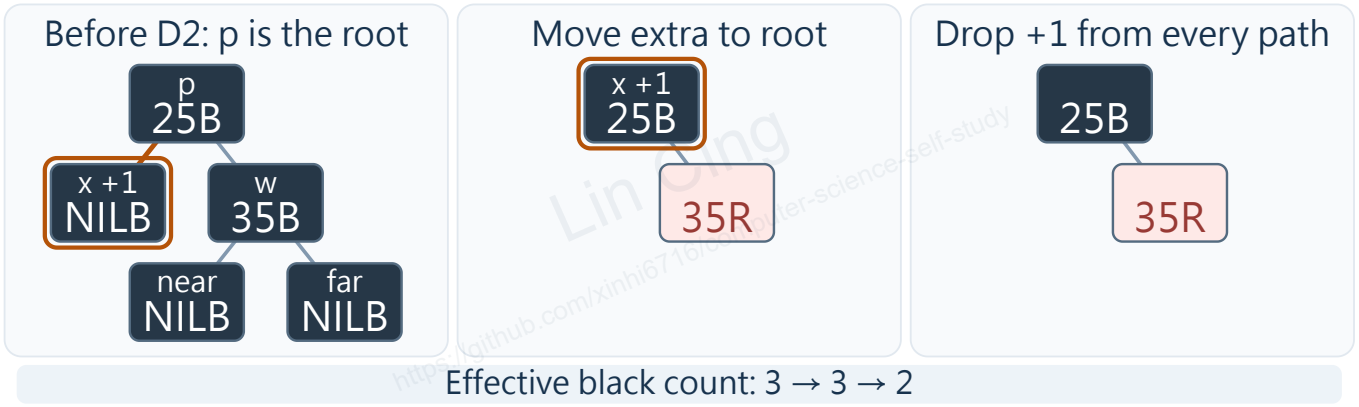
Continue the example: w35B has NIL children. Make35 red and move x from NIL to25R. The loop exits and25 becomes black. Finally45B has left25B(right35R) and right55B(right65R).

D2 moves the deficit upward or absorbs it at a red parent (continued)



D2 at the root reduces every path by one black

Start with25B and black leaves15B,35B. Deleting15 leads D2 to make35 red and move x to root25. Dropping extra from all root-leaf paths leaves25B(right35R). Root bh decreases from2 to1, with equal path counts. This differs from absorption by blackening a red parent.



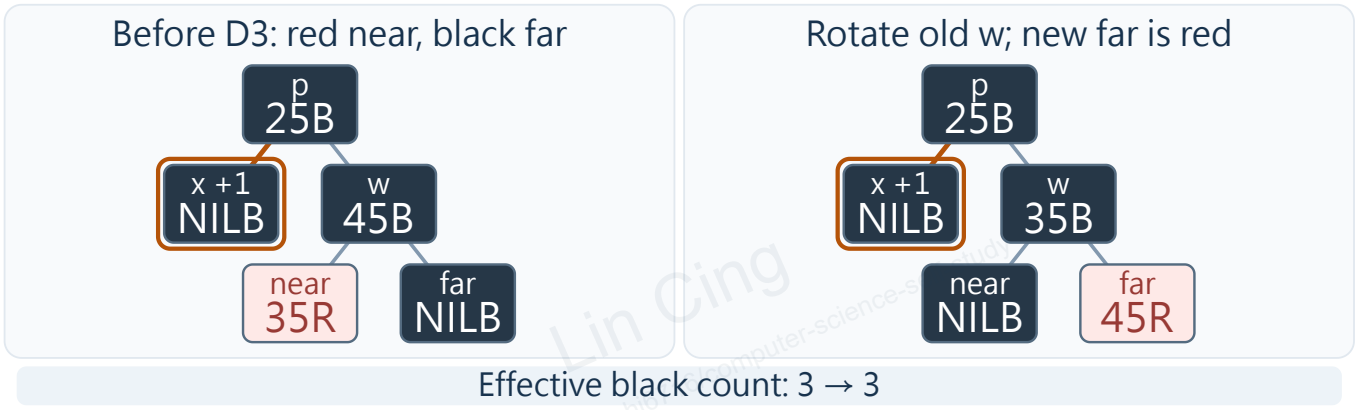
D3 moves an available red node to the far side

w is black, near N is red, and far is black. Let x's actual count be b. Both w children contribute b; red N's children each contribute b, as does far. Make N black and w red, then right-rotate w. New black sibling N has old N.left contributing b and old red w whose old N.right and far also contribute b. Its total remains b+1, preserving everything above p.

New sibling's far child is old red w, so execute D4 immediately. Extra remains at x; this case neither moves the deficit upward nor terminates.

Insert15,25,45,35 to obtain25B with left15B and right45B(left35R). Delete15, leaving x=NIL, w45B, near35R, and black far=NIL. Make35 black and45 red, right-rotate45, and obtain25B with right35B(right45R). D4 must follow.

D3 moves an available red node to the far side (continued)



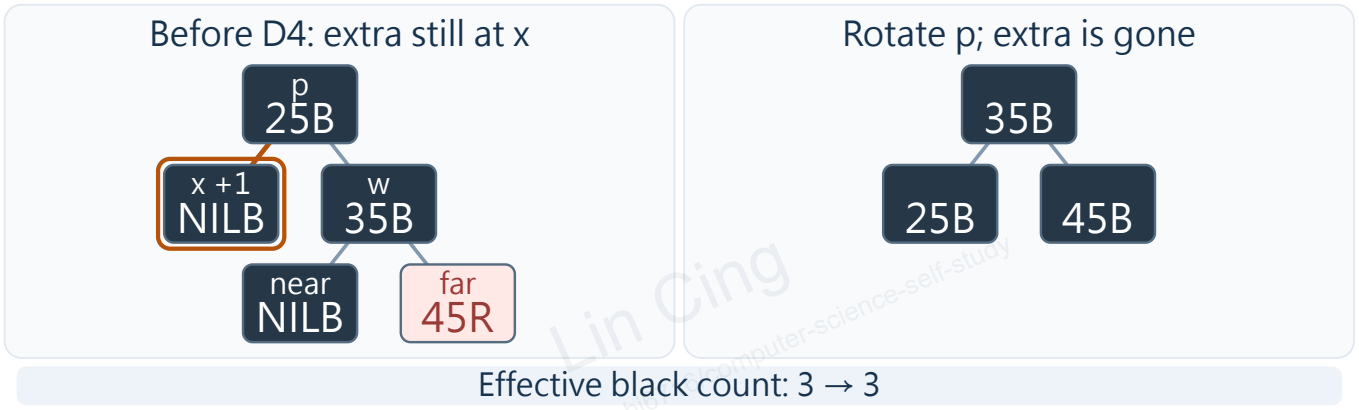
D4 repairs both sides by recoloring and rotation

Let p's original black contribution be c∈{0,1}, x's actual count b, and both children of black w contribute b; far is red. Give w p's old color, make p and far black, and left-rotate p. New root w has black left child p with equal actual b on x and near, so its complete left count is b+1. Far changes red to black and also contributes b+1. New local total is c+b+1, equal to original p's effective c+(b+1).

If new w is red, its children p and far are black. Making p black creates no red-red conflict, and w retains p's external color relationship. Extra is eliminated. Set x=root to stop and ensure the root is black.

Continue D3:35 takes25's old black color;25 and far45 become black. Left-rotate25 to obtain35B with25B,45B children. Inorder is25,35,45, and every root-NIL path contains3 blacks with no extra.

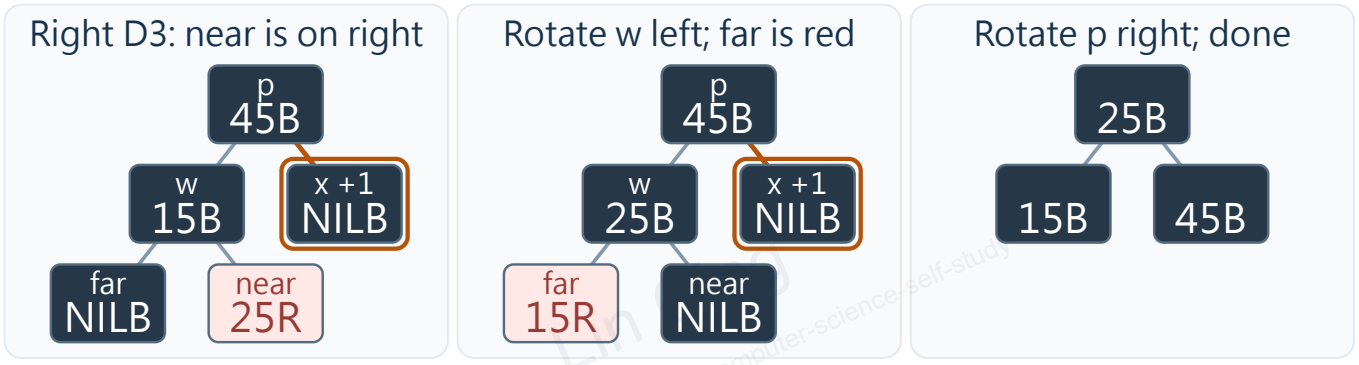
D4 repairs both sides by recoloring and rotation (continued)



Effective black count: 3 → 3

Mirror example: insert15,35,45,25, then delete35. Direct successor45B becomes root; x=NIL is45.right, so NIL.parent must be45. Sibling15B has near25R and far=NIL. Right-side D3 left-rotates15, then right-side D4 right-rotates45, yielding25B with15B,45B children.

D4 repairs both sides by recoloring and rotation (continued)



Effective black count: 3 → 3 → 3

Count the black nodes along the same D3→D4 example

Every row **includes its starting node and NIL**. D3 leaves x=NIL in place; D4 removes the extra black.

Stage and path	Actual black	Extra	Effective total
Before D3: 25B → NIL(x)	2	1	3
Before D3: 25B → 45B → 35R → NIL	3	0	3
After D3: 25B → 35B → 45R → NIL	3	0	3
After D4: 35B → 25B → NIL	3	0	3
After D4: 35B → 45B → NIL	3	0	3

Each black node adds 1 and each red node adds 0. Add the extra only on paths through x. The other NIL paths also total 3 by the same rule; check them in the figures. After D4 both sides have the needed black without an extra.

From local conservation to the general proof and cost

BST relocation preserves order and every undeleted record. Removing red directly leaves a valid tree. Removing black assigns extra to x so effective ancestor-to-leaf counts remain balanced. D1 and D3 preserve its position and effective counts; D2 moves the sole extra upward; D4 eliminates it. A red x is repaired by final blackening. In D2, newly red w has black children; rotations and recolorings satisfy the checked local color conditions. At termination all five properties hold.

D2 ascends one level with no rotation, at most O(log(n+1)) times. After D1, p is red, so D2 absorbs, or D3/D4 terminates. Therefore D1, D3, and D4 occur at most once each, for at most3 rotations. Searching for z and its successor plus relocation and repair totals O(log(n+1)). NIL, root, and red-child absorption are valid base cases; deleting the last node leaves black root=NIL.

This proof covers every legal red-black tree. Finite example programs additionally check global key order, reverse parent links, black children of red nodes, equal NIL-path black counts, root color, and undeleted record identities. They cover both sides of all insertion and deletion cases. Recoloring may follow the height; at most3 rotations does not make complete deletion O(1).

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 13.

Lin Cing, personal handwritten data-structures notes on red-black and2-3-4 trees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study